OLYMPIX

SECURITY VERIFICATION REPORT

Fables

Automated formal invariant verification

A formal-verification engagement against **Fables** — Olympix translated 78 protocol invariants into symbolic Foundry checks, drove the halmos prover over every one, and recorded a passing result for 62 of them under the setup, assumptions and bounds each check declares — with a runnable artifact behind every verdict.

INVARIANTS	CHECKS PASSED	FINDINGS	INCONCLUSIVE	DURATION
78	**62**	**4**	**1**	**6h 32m**

Contents

01 **Executive summary**

02 **Why formal verification matters**

03 **The protocol in one picture**

04 **Scope at a glance**

05 **How the engine works**

06 **Verification results**

07 **Deliverables & next steps**

A **Appendix — full invariant table**

01 Executive summary

Olympix ran its formal invariant-verification engine against the Fables protocol, turning its documented safety properties into machine-checked symbolic checks.

78
protocol invariants taken into verification across 14 contract(s)

62
invariants whose generated check passed under its recorded setup, assumptions and bounds

4
findings — invariants falsified with a concrete counterexample

99%
of invariants successfully translated into symbolic Foundry checks

Formal invariant verification
78 invariants · 99% translated · 1 inconclusive · 10 vacuous · 1 untranslatable

62 checks passed · 4 findings
every check and counterexample ships as a runnable Foundry test

Engagement at a glance

Target	Fables
Prover	`halmos 0.3.3`
Run date	November 11, 2026 UTC
Duration	6h 32m wall-clock
Mutation sensitivity check	Enabled for some checks — see per-invariant sensitivity status
Scope	78 invariant(s) across 14 contract(s)
Run	session `3ad548de-4adf-4c21-bfd4-80141c63f9f3`

02 Why formal verification matters

Symbolic checks, not sampled test runs

Testing shows the presence of bugs on the inputs it happened to try; a symbolic check covers every input **within the domain the check declares** at once. The 62 checks that passed did so over the symbolic inputs, mocks, assumptions and bounds recorded with each one — which is narrower than the original natural-language invariant. Read each check's declared scope before relying on it.

Counterexamples are concrete findings

When a generated check can be broken, the prover returns a falsifying assignment for that check. This run produced 4 such counterexample(s), each reproduced verbatim in section 06 with the runnable test that triggers it. Each one must be replayed against the generated harness before it is treated as a defect in the contract.

Sensitivity is measured separately, where it was measured at all

A check over an unreachable state space establishes nothing. Where the sensitivity check ran, the check was additionally run against deliberately altered contract source; one that still passes on altered code is reported as low sensitivity. A skipped or errored sensitivity run is unassessed — it is never evidence that a check is meaningful.

Fast enough for every release

Specification to verdict took 6h 32m for 78 invariants. This can run on every release candidate, not once per audit cycle.

03 The protocol in one picture

The contracts below carry Fables's documented invariants; contracts that share invariants share state and trust assumptions.

FablesLedgerV2
40 invariants
High

LedgerLibV2
29 invariants
High

FablesFeeEngine
24 invariants
High

FablesStableFee
22 invariants
High

FablesStableHookLite
13 invariants
High

FablesFeeFull
11 invariants
High

FeeRef
11 invariants
High

FablesNativeETHV2
8 invariants
High

How they connect



External dependencies

FablesExclusiveLiquidityV2, FablesFeeEngine, FablesFeeFull, FablesStableFee, FablesHook → **Uniswap V4 PoolManager**

FablesHookCore, FablesLedgerV2, LedgerLibV2, FablesFeeEngine, FablesFeeFull → **Uniswap V4 PoolManager (v4-core)**

FablesHookCore, FablesFeeEngine, FablesLedgerV2, FablesStableFee, FablesStableHookLite → **OpenZeppelin AccessManager (authority)**

FablesLedgerV2, FablesLedgerV2.stake, FablesLedgerV2.syncRanges, FablesLedgerV2.setBallotGate → **Fables Voter / Ballot Gate (IBallotGate + ve escrow)**

FablesLedgerV2, LedgerLibV2, FablesNativeETHV2, LedgerLibV2.sweepDistributor, LedgerLibV2.validateDistributor → **Fables Distributor (IDistributorFables) and FABLES fee sink**

FablesNativeETHV2, FablesHookETH, FablesHookETHExclusive, FablesLedgerV2._sweepDistributor, FablesLedgerV2._sweepClaimFee → **WETH9**

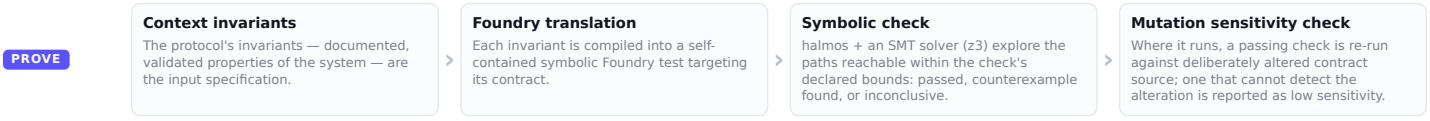
04 Scope at a glance

Every invariant is anchored to a specific contract. The table below shows how the verified surface distributes across the protocol.

Contract	Invariants	Criticality mix	Verdict mix
FablesLedgerV2	27	17 High · 10 Medium	23 check passed · 4 passed, low sensitivity
FablesFeeEngine	17	9 High · 7 Medium · 1 Low	10 check passed · 4 violated · 2 passed, low sensitivity · 1 untranslatable
LedgerLibV2	10	4 High · 6 Medium	7 check passed · 3 passed, low sensitivity
FablesStableFee	8	4 High · 4 Medium	6 check passed · 1 inconclusive · 1 passed, low sensitivity
FablesStableHookLite	3	2 High · 1 Medium	3 check passed
FablesFeeFull	2	2 High	2 check passed
DiscountLib	2	2 Medium	2 check passed
FablesHookCore	2	1 High · 1 Medium	2 check passed
AccWrapHarness	2	2 High	2 check passed
FeeRef	1	1 Medium	1 check passed
LedgerHandler	1	1 Low	1 check passed
ConservationHandler	1	1 Low	1 check passed
IsolationHandler	1	1 Low	1 check passed
EngineInvariantHandler	1	1 Low	1 check passed
Total	**78**		**62 proven · 4 findings**

05 How the engine works

The engine starts from the protocol's documented invariants, compiles each one into a symbolic Foundry proof obligation, and hands it to an SMT-backed prover.

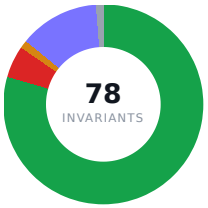


A passing check is a statement about the generated test under its recorded setup, mocks, assumptions and bounds, with a separately reported sensitivity status where that check ran — not a proof of the original invariant over all reachable states.

06 Verification results

Every invariant received exactly one verdict. **Check passed** means the generated check ran to completion and the solver found no counterexample within that check's declared scope; **violated** means the solver constructed a falsifying assignment for that check, which still requires replay against the harness. Neither verdict certifies the original natural-language invariant.

78 INVARIANTS · VERDICT BREAKDOWN



Check passed	62 · 79%
Violated	4 · 5%
Inconclusive	1 · 1%
Passed, low sensitivity	10 · 13%
Untranslatable	1 · 1%
Unsupported property kind	0 · 0%

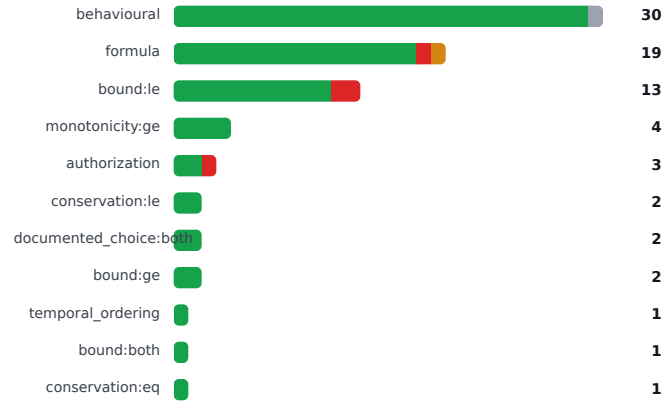
99% of invariants were successfully translated into runnable symbolic proofs before proving began.

SCOPE BY CRITICALITY



High	41
Medium	32
Low	5

VERDICT BY INVARIANT TYPE



What these verdicts do and do not establish

- A passing check is a result about the **generated test** — its setup, mocks, symbolic domains, assumptions and bounds — not a proof of the original natural-language invariant over all reachable states.
- A finite call sequence from one concrete initial state does not establish that a property is preserved from every state satisfying it.
- Counterexamples are falsifying assignments for the generated check and require replay against that harness before being treated as contract defects.
- Mutation sensitivity was measured only where reported. A skipped or errored sensitivity run is unassessed, never evidence that a check is meaningful.
- Liveness and temporal properties are not verified by this engine; an invariant labelled as such received no temporal reasoning.

Checks that passed

FablesLedgerV2

- `inv_001_FablesLedgerV2` `withdraw` must succeed for a holder of receipts regardless of any role action, pause or configuration. Code: `withdraw` has no `whenNotPaused`/`whenPoolNotPaused` modifier and no role gate; `_withdrawTo` syncs the range (`_syncRange`), settles the user (`settleUser`, with saturating `_accrue` and unchecked `forgone` sums), auto-unstakes any shortfall from staked shares, burns receipts and removes liquidity through `_modify`; fees are credited, not paid. Admin-settable state it touches includes the claim-fee history walk (`claimFeeFor`), claim-fee recipient, distributor/sinks and the ballot gate; none of it may make `withdraw` revert. `withdrawAndClaim` is pausable and is not the guaranteed exit.
  check `check_inv_001_FablesLedgerV2` · deployed as `LedgerHarness`, a concrete stand-in for the target · 9 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_002_FablesLedgerV2` The ledger never owes more than it holds.
  check `check_inv_002_FablesLedgerV2` · deployed as `LedgerV2CreditHarness`, a concrete stand-in for the target · 167 path(s) explored · sensitivity unassessed
- `inv_003_FablesLedgerV2` Fees earned before a deposit, stake or transfer must not be credited to the newcomer. Mechanism: every share/stake change (`deposit`, `_withdrawTo`, `stake`, `unstake`, `transfer`, `transferFrom`) first calls `_syncRange(id)` (collects v4 fees at current totals via `creditFees`/`_split`) and then `_settleUser` for each affected account, which sets the account's `checkpoint*X128`/`stakedCheckpoint*X128` to the current accumulator levels before the balance changes. A recreated range keeps old accumulator levels; the new depositor's checkpoint is set to them.
  check `check_inv_003_FablesLedgerV2` · deployed as `FablesHook`, a concrete stand-in for the target · 470 path(s) explored · sensitivity unassessed
- `inv_013_FablesLedgerV2` PAUSER can pause deposits, claims, stakes, transfers and revenue sweeps for up to 7 days per call, clear pokes, switch off the continuation fee and revoke discounts.
  check `check_inv_013_FablesLedgerV2` · deployed as `LedgerHarness`, a concrete stand-in for the target · 3 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_015_FablesLedgerV2` LP principal entry: `FablesLedgerV2.deposit(key, tickLower, tickUpper, liquidity, amount0Max, amount1Max, deadline)` registers `_rangeKeys[id]` (id = keccak256(abi.encode(poolId, tickLower, tickUpper))) on first use, calls `_syncRange(id)` and `_settleUser(id, msg.sender)`, increments `_ranges[id].resetSeq` if `totalShares` was 0, raises `_ranges[id].totalShares` by `liquidity`, mints `liquidity` ERC-6909 receipts to `msg.sender`, then `_modify` opens a PoolManager unlock in which `LedgerLibV2._onModify` calls `modifyLiquidity(+liquidity, salt 0)` on the hook-owned position and `_settlePrincipal` pulls `principal = callerDelta - feesAccrued` per currency: ERC20 via `safeTransferFrom(msg.sender, poolManager, owed)` + `settle()`, native via `settle{value: owed}()` from the hook's balance. `_checkBound` reverts if paid principal exceeds `amount0Max`/`amount1Max`. On native pools `_refundNative` sends `address(this).balance - preBalance` back to `msg.sender` last.
  check `check_inv_015_FablesLedgerV2` · deployed as `LedgerHarness`, a concrete stand-in for the target · 4 path(s) explored · sensitivity unassessed
- `inv_016_FablesLedgerV2` Withdrawing and unstaking are never pausable or gated.
  check `check_inv_016_FablesLedgerV2` · deployed as `LedgerHarness`, a concrete stand-in for the target · 2 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_020_FablesLedgerV2` The claim fee can be set, at most 20% of LP fees.
  check `check_inv_020_FablesLedgerV2` · deployed as `LedgerHarness`, a concrete stand-in for the target · 3 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_027_FablesLedgerV2` Holder fee payout: `claimFees(key, tickLower, tickUpper, to, maxFeeBps)` (whenPoolNotPaused) reverts `NothingToClaim` when the caller has zero balance and zero owed, runs `_syncRange` + `_settleUser`, zeroes `position.owed0/owed1`, then `_payout` opens a PAYOUT unlock where `LedgerLibV2._redeem` does `poolManager.burn(hook, currency.toId(), amount)` and `poolManager.take(currency, to, amount)` for each currency. `withdrawAndClaim` does the same after `_withdrawTo`. The hook's PoolManager claim balance (minted in `creditFees`) is the source of every payout.
  check `check_inv_027_FablesLedgerV2` · deployed as `LedgerHarness`, a concrete stand-in for the target · 2 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_042_FablesLedgerV2` A transfer of 0 reverts; staked receipts must be unstaked before they move.
  check `check_inv_042_FablesLedgerV2` · deployed as `LedgerHarness`, a concrete stand-in for the target · 3 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_043_FablesLedgerV2` Sink changes: `setClaimFee`, `setDistributor` and `setFFFeeSink` first call `_settleSinks(settleFirst)`, paying the current sinks the booked pots (`distributorOwed` / `claimFeeOwed`) of the listed currencies; everything else, including fees not yet synced into the ledger, is paid to the sinks set when it is swept. Sweeps (`sweepDistributor`, `sweepClaimFee`) are permissionless, pay `distributorOwed[currency]` to `ffFeeSink` when currency == `distributorFF` else to `distributor`, and `claimFeeOwed` to `claimFeeRecipient`; a sweep to a zero target is a no-op keeping the accrual. Native revenue is paid in WETH. The claim-fee recipient is hook-wide (any pool's `setClaimFee` rewrites it for all pools) and never returns to zero once set.
  check `check_inv_043_FablesLedgerV2` · deployed as `LedgerHarness`, a concrete stand-in for the target · 1 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_045_FablesLedgerV2` Shared state `distributorOwed[currency]` and `claimFeeOwed[currency]` (hook-wide per currency, summed across all pools/ranges): raised only inside `LedgerLibV2._split` (from `creditFees`, reached by every `_syncRange`/`_modify` path: deposit, withdraw, withdrawAndClaim, claimFees, stake, unstake, transfer, transferFrom, syncRanges); reset to 0 by `sweepDistributor`/`sweepClaimFee` (anyone, when not paused hook-wide and sink set) and by `_settleSinks` inside `setClaimFee`, `setDistributor`, `setFFFeeSink`. Holder `position.owed0/owed1` is raised by `settleUser` and reset to 0 by `claimFees` and `withdrawAndClaim`. All three are paid from the same pool of hook-held PoolManager claims per currency.
  check `check_inv_045_FablesLedgerV2` · deployed as `LedgerHarness`, a concrete stand-in for the target · 1 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_046_FablesLedgerV2` Shared state claim-fee policy: `claimFeeRecipient` is hook-wide while `_poolClaimFee[poolId]` (bps, history, allTimeMin) is per pool. `setClaimFee` (sweeps `settleFirst`, then pushes a history entry `{fromTs, effective bps}` for one pool and overwrites the hook-wide `claimFeeRecipient`; reverts when trying to set recipient back to 0 once set) and `setClaimFeeBps` (requires a recipient, pushes an entry for one pool, no sweep) are the only writers; `recordClaimFee` rejects recipient == hook, == `distributor`, == `ffFeeSink`. `claimFeeSyncIndex[id]` is written by `creditFees` (every non-idle sync) and by `_syncRange`'s idle branch, always to the pool's current history length.
  check `check_inv_046_FablesLedgerV2` · deployed as `LedgerHarness`, a concrete stand-in for the target · 2 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_047_FablesLedgerV2` unstake is never pausable, so a holder can always stop routing fees to the distributor.
  check `check_inv_047_FablesLedgerV2` · deployed as `LedgerHarness`, a concrete stand-in for the target · 2 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_049_FablesLedgerV2` `FablesLedgerV2._poolClaimFee[poolId]` (bps, history, allTimeMin): set by ADMIN via `FablesLedgerV2.setClaimFee` and by REVENUE_OPS via `FablesLedgerV2.setClaimFeeBps`, bounded by `LedgerLibV2.recordClaimFee` to `bps <= MAX_CLAIM_FEE_BPS` (2000), bps != 0 requires a non-zero recipient, plus `_checkKey`; each call pushes (fromTs, effective bps) to history and lowers allTimeMin if smaller. setClaimFeeBps reverts while claimFeeRecipient == 0. Read by `LedgerLibV2.creditFees` -> `LedgerLibV2.claimFeeFor` (rate = min over history entries since the range's `claimFeeSyncIndex`, starting from current bps, walking at most 32 entries, else min with allTimeMin; 0 if sinceIndex == 0 or recipient == 0) and `LedgerLibV2._split` (claim cut c = net_unstaked_fee * rate / 10000, rounded down, booked to claimFeeOwed); `FablesLedgerV2._effectiveClaimFee` compared to the user's maxFeeBps in `claimFees` / `withdrawAndClaim`.
  check `check_inv_049_FablesLedgerV2` · deployed as `ClaimFeePolicyHarness`, a concrete stand-in for the target · 9 path(s) explored · sensitivity unassessed
- `inv_052_FablesLedgerV2` `FablesLedgerV2.ffFeeSink`: set by ADMIN via `FablesLedgerV2.setFFFeeSink` after sweeping `settleFirst` currencies; bounded to zero, or a non-zero address `!= distributor`, `!= claimFeeRecipient`, `!= address(this)`, `!= poolManager`. Read by `LedgerLibV2.sweepDistributor` (destination of `distributorOwed[distributorFF]`, i.e. FABLES-denominated staked-share fees; zero = accrual waits), `LedgerLibV2.recordClaimFee` and `LedgerLibV2.validateDistributor` (exclusions). The native override `FablesNativeETHV2._sweepDistributor` does not read it.
  check `check_inv_052_FablesLedgerV2` · deployed as `LedgerHarness`, a concrete stand-in for the target · 5 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_054_FablesLedgerV2` Shared state pause timers: `pausedUntil` is set to `now + duration` or lifted to 0 by `setPaused(duration <= 7 days)`, and only extended by `pause(duration)` (reverts if `now + duration <= pausedUntil`); `pausedUntilOf[poolId]` likewise by `setPoolPaused` (set/lift) and `pausePool` (extend only). `pausedFor(poolId)` gates `deposit`, `withdrawAndClaim`, `claimFees`, `stake`, and (via range key) `transfer`/`transferFrom`; hook-wide `paused()` alone gates `sweepDistributor`/`sweepClaimFee`. `withdraw`, `unstake` and `syncRanges` read neither.
  check `check_inv_054_FablesLedgerV2` · deployed as `LedgerHarness`, a concrete stand-in for the target · 10 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_056_FablesLedgerV2` `FablesLedgerV2.pausedUntil` (hook-wide): set by ADMIN via `FablesLedgerV2.setPaused` (duration <= MAX_PAUSE = 7 days; 0 sets it to 0; may shorten) and by PAUSER via `FablesLedgerV2.pause` (1 <= duration <= 7 days, new until = now + duration must exceed the current pausedUntil; no cap on how many times it can be extended). Read by `FablesLedgerV2.paused` / `FablesLedgerV2.pausedFor` -> `whenNotPaused` (`sweepDistributor`, `sweepClaimFee`), `whenPoolNotPaused` (`deposit`, `claimFees`, `withdrawAndClaim`, `stake`) and `whenRangeNotPaused` (`transfer`, `transferFrom`). Not read by `withdraw`, `unstake`, `syncRanges`, or the internal sweeps run by `setClaimFee`/`setDistributor`/`setFFFeeSink` via `_settleSinks`.

check `check_inv_056_FablesLedgerV2` · deployed as `LedgerHarness`, a concrete stand-in for the target · 7 path(s) explored · sensitivity assessed: detected a deliberate change

- `inv_057_FablesLedgerV2` `FablesLedgerV2.pausedUntilOf[poolId]` (per-pool): set by ADMIN via `FablesLedgerV2.setPoolPaused` (duration <= 7 days, 0 lifts, may shorten) and by PAUSER via `FablesLedgerV2.pausePool` (1..7 days, only extends past the current value), both requiring `_checkKey(key)`. Read by `FablesLedgerV2.pausedFor` -> `whenPoolNotPaused` (`deposit`, `claimFees`, `withdrawAndClaim`, `stake`) and `whenRangeNotPaused` (`transfer`, `transferFrom` via the range's stored key; an unknown range resolves to the zero key). Not read by sweeps, `withdraw`, `unstake`, `syncRanges`.
  check `check_inv_057_FablesLedgerV2` · deployed as `LedgerHarness`, a concrete stand-in for the target · 6 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_058_FablesLedgerV2` `FablesLedgerV2.ballotGate`: set by ADMIN via `FablesLedgerV2.setBallotGate`, bounded to zero, or a contract with code that answers `ballotBacks`, `isGaugeEarning`, `gaugeWeightNow` without reverting and whose `ve()` escrow (non-zero, with code) returns `voter() == gate`. Read by `FablesLedgerV2.stake` (reverts GaugeNotEarning if gate == 0 or `isGaugeEarning(poolId)` is false; compares `gaugeWeightNow` to the caller's minGaugeWeight) and `FablesLedgerV2.syncRanges` (reverts BallotGateUnset if zero; requires `ballotBacks(msg.sender, tokenId, poolId)` per staked range).
  check `check_inv_058_FablesLedgerV2` · deployed as `FablesHook`, a concrete stand-in for the target · 2 path(s) explored · sensitivity unassessed
- `inv_062_FablesLedgerV2` Fee crediting in a share change occurs at post-mutation totals, which the quote says is safe only because each caller's earlier _syncRange drained the fees.
  check `check_inv_062_FablesLedgerV2` · deployed as `JitLedgerHarness`, a concrete stand-in for the target · 9 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_066_FablesLedgerV2` The claim-fee rate charged on unstaked LP fees is at most 20% (`MAX_CLAIM_FEE_BPS` = 2000), enforced in `LedgerLibV2.recordClaimFee` for both `FablesLedgerV2.setClaimFee` (ADMIN) and `FablesLedgerV2.setClaimFeeBps` (REVENUE_OPS); the rate applied in `LedgerLibV2._split` is `claimFeeFor` over recorded history entries.
  check `check_inv_066_FablesLedgerV2` · deployed as `LedgerHarness`, a concrete stand-in for the target · 4 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_067_FablesLedgerV2` `FablesLedgerV2.withdraw` and `FablesLedgerV2.unstake` carry no pause modifier and no role gate (withdraw auto-unstakes the shortfall), whereas `withdrawAndClaim`, `claimFees`, `deposit`, `stake` and receipt transfers are gated by `whenPoolNotPaused` / `whenRangeNotPaused`. The ledger NatSpec documents this as a design property.
  check `check_inv_067_FablesLedgerV2` · deployed as `FablesHook`, a concrete stand-in for the target · 2 path(s) explored · sensitivity unassessed
- `inv_069_FablesLedgerV2` Each pause call extends a freeze by at most `MAX_PAUSE` (7 days) from the call's timestamp: `FablesLedgerV2.pause` / `pausePool` (PAUSER) require 1 <= duration <= 7 days and until > current value; `setPaused` / `setPoolPaused` (ADMIN) require duration <= 7 days. The cap is per call; the code sets no limit on the number of successive extensions.
  check `check_inv_069_FablesLedgerV2` · deployed as `LedgerHarness`, a concrete stand-in for the target · 10 path(s) explored · sensitivity assessed: detected a deliberate change

FablesFeeEngine

- `inv_007_FablesFeeEngine` Pokes move each side's fee within [max(floor, 50% of base), min(pool cap, category cap)] for at most 72 h, raising at most one step per 60 s.
  check `check_inv_007_FablesFeeEngine` · deployed as `Engine`, a concrete stand-in for the target · 32 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_014_FablesFeeEngine` ADMIN powers include setting the claim fee, but the quote only names moving LP principal, setting a fee above 10% and un-configuring a pool as listed limits.
  check `check_inv_014_FablesFeeEngine` · deployed as `FablesFeeEngineHarness`, a concrete stand-in for the target · 1 path(s) explored · sensitivity unassessed
- `inv_029_FablesFeeEngine` Pokes move each side's fee within [max(floor, 50% of base), min(pool cap, category cap)] for at most 72 h, raising at most one step per 60 s.
  check `check_inv_029_FablesFeeEngine` · deployed as `TestEngine`, a concrete stand-in for the target · 8 path(s) explored · sensitivity unassessed
- `inv_030_FablesFeeEngine` Pool fee configuration bounds: `setCategory` requires `capMax <= ABSOLUTE_MAX_FEE (100_000 = 10%)`, `floorMin <= capMax`, `raiseStep != 0`; `configurePool` requires dynamic-fee key with `hooks == this`, an existing category, `1 <= cap <= capMax`, `max(base0, base1) <= cap`, `floorMin <= floor <= min(base0, base1)`, writes bases/floor/cap into `_word[id]` and `maxFee[id] = cap` (kept pokes, CF and filter). Existing pools keep stored bounds when a category is later changed. ADMIN may move bases anywhere inside [floor, cap] with no raiseStep limit.
  check `check_inv_030_FablesFeeEngine` · deployed as `FeeEngineH`, a concrete stand-in for the target · 6 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_037_FablesFeeEngine` `FablesFeeEngine._word[id]` bases/floor/cap, `FablesFeeEngine._meta[id].category` and `FablesHookCore.maxFee[id]`: set by ADMIN via `FablesFeeEngine.configurePool`, bounded to `key.hooks == this`, dynamic-fee key, existing category, `c.floorMin <= floor <= min(base0, base1)`, `max(base0, base1) <= cap <= c.capMax`, `cap >= 1`; `maxFee[id] = cap`. Callable again on an initialized pool (reconfigure keeps pokes, Bloom filter and CF state). Read by the `FablesFeeFull` fallback (beforeSwap fee), `FablesFeeFull.currentFee`, `FablesFeeFull.feeFor`, `FablesFeeEngine.pokeFee` (floor/cap bounds), discount floor; `maxFee` read by `FablesLedgerV2._checkKey` (deposit, withdraw, withdrawAndClaim, claimFees, stake, setClaimFee, setPoolPaused, pausePool) and `FablesFeeEngine.initializePool`.
  check `check_inv_037_FablesFeeEngine` · deployed as `Engine`, a concrete stand-in for the target · 6 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_039_FablesFeeEngine` `FablesFeeEngine._cf[id]` (mode, kQ8, cfCap, band16, tau/decay1, period, maxAge, anchor, tLast) and the `cfOn` bit of `FablesFeeEngine._word[id]`: parameters set by ADMIN via `FablesFeeEngine.configureCF`, bounded to pool configured, CF off, `mode <= 1`, `1 <= kQ8 <= 256`, `1 <= cfCap <= 2500`, `band16 <= 800`, `5 <= tau <= 300`, `1 <= period <= 60`, `period < min(255, 5*tau)`; anchor = current tick * 256. Switched on (re-anchored at current tick in [minTick, maxTick], before deadline, kQ8 != 0) or off by FEE_POKER via `FablesFeeEngine.setCFActive`; killed (off, kQ8 = 0) by PAUSER via `FablesFeeEngine.killCF`. Read by the `FablesFeeFull` fallback when cfOn: anchor A decays toward T = tick*256 by decay1^dt when dt >= period (snap at maxAge) and is written back; x = (zeroForOne ? A - T : T - A), negated in mode 1, minus band16*16; if x > 0, fee = min(fee + min((kQ8 * x * 100) >> 16, cfCap), cap).
  check `check_inv_039_FablesFeeEngine` · deployed as `CFHarness`, a concrete stand-in for the target · 8 path(s) explored · sensitivity unassessed
- `inv_048_FablesFeeEngine` No hook uses return-delta flags, remove-liquidity or donate callbacks, or hookData.
  check `check_inv_048_FablesFeeEngine` · deployed as `FablesFeeEngineHarness`, a concrete stand-in for the target · 2 path(s) explored · sensitivity unassessed
- `inv_065_FablesFeeEngine` ADMIN is documented as unable to move LP principal, set a fee above 10%, or un-configure a pool.
  check `check_inv_065_FablesFeeEngine` · deployed as `FeeEngineHarness`, a concrete stand-in for the target · 6 path(s) explored · sensitivity unassessed
- `inv_070_FablesFeeEngine` A FEE_POKER poke is valid for at most `MAX_POKE_TTL` (72 h) and a raising poke may move the resolved keeper fee up by at most the category `raiseStep`, no sooner than `RAISE_INTERVAL` (60 s) after the pool's last raise (`FablesFeeEngine.pokeFee`, `FablesFeeEngine._raises`, `FablesFeeEngine._meta.lastRaiseAt`).
  check `check_inv_070_FablesFeeEngine` · deployed as `FablesFeeEngineHarness`, a concrete stand-in for the target · 186 path(s) explored · sensitivity unassessed
- `inv_040_FablesFeeEngine` SEEDER may only initialize a pool the admin has already configured (FablesHook builds), at the price it passes: `FablesFeeEngine.initializePool(key, sqrtPriceX96)` requires `maxFee[id] != 0` (set only by `configurePool`) and native currency0 only on native builds; the hook places no bound on `sqrtPriceX96`. SEEDER must not be able to configure pools or change existing ones (v4 reverts re-initialization of an initialized pool).
  check `check_inv_040_FablesFeeEngine` · deployed as `SeederHarness`, a concrete stand-in for the target · 1 path(s) explored · sensitivity unassessed

LedgerLibV2

- `inv_017_LedgerLibV2` Fee collection and split (`LedgerLibV2.creditFees` → `_split`): `_syncRange(id)` (skipped when `_rangeKeys[id].set` is false or `totalShares == 0`, which only rolls `claimFeeSyncIndex[id]`) runs `modifyLiquidity(0)` on the range; `feesAccrued` (fee0, fee1) are minted as PoolManager ERC-6909 claims to the hook. With `total = range.totalShares`: if `total == 0` all fees go to `distributorOwed`; else `stakedCut = _takeStakedCut(fee, total)` goes to `distributorOwed` and raises `accStaked*X128 += cut * 2^128 / totalStaked`; if `unstakedTotal = total - totalStaked == 0` the remainder also goes to `distributorOwed`; otherwise `net = fee - stakedCut`, claim fee `c = net * feeBps / 10_000` (only when `feeBps != 0` and `claimFeeRecipient != 0`) goes to `claimFeeOwed`, and `accFee*X128 += (net - c) * 2^128 / unstakedTotal` (unchecked, wraps). Inputs: fee0/fee1 grow with any swap in the pool (any user), `totalShares`/`totalStaked` move via deposit/withdraw/stake/unstake (any holder), `feeBps` via REVENUE_OPS/ADMIN setters.
  check `check_inv_017_LedgerLibV2` · deployed as `SplitHarness`, a concrete stand-in for the target · 15 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_018_LedgerLibV2` Staked cut with carried remainder: `LedgerLibV2._takeStakedCut` computes `carried = stakedResidual + mulDiv(mulmod(fee, totalStaked, total), 2^128, total)` and `cut = mulDiv(fee, totalStaked, total) + (carried >> 128)`, storing `uint128(carried)` back into `range.stakedResidual0/1`. The unstaked side floors twice (`_split` comment: dust under one unit per collection stays as hook-held claims). `totalStaked` and `total` are moved by any holder through `stake`, `unstake`, `deposit`, `withdraw` before the next sync.
  check `check_inv_018_LedgerLibV2` · deployed as `CreditHarness`, a concrete stand-in for the target · 197 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_021_LedgerLibV2` The claim fee is charged at no more than the lowest rate in force since a range's last sync, and its recipient is hook-wide.
  check `check_inv_021_LedgerLibV2` · 4 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_022_LedgerLibV2` The claim fee is charged at no more than the lowest rate in force since a range's last sync.
  check `check_inv_022_LedgerLibV2` · deployed as `ClaimFeeHarness`, a concrete stand-in for the target · 10 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_024_LedgerLibV2` The staked cut carries its remainder to the next collection, and the claim fee rounds down in LPs' favour.
  check `check_inv_024_LedgerLibV2` · 93 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_025_LedgerLibV2` A claim-fee cut applies to the whole backlog; a raise only to later ones.
  check `check_inv_025_LedgerLibV2` · 6 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_072_LedgerLibV2` Owed accrual is saturating, with tokens above 2^127 units unsupported, because a revert would block the holder's withdraw.
  check `check_inv_072_LedgerLibV2` · deployed as `address`, a concrete stand-in for the target · 18 path(s) explored · sensitivity assessed: detected a deliberate change

FablesStableFee

- `inv_032_FablesStableFee` Stable builds: the charged fee must be >= StablePair's fee for the same state capped at 10% (i.e. >= min(StablePair fee, 100_000 pips)) — never below it, possibly above it. Once the price ratio to the reference is below 0.9, only the decay target is floored at its value at 0.9 (`_targetFee`); the formula must read and store the real (unclamped) AMM price (as StablePair does) so the movement adjustment in `_decayStartFee` uses real prices; the floor only ever raises the target. (Fix for pre-audit findings 5.1.1 a/b.)
  check `check_inv_032_FablesStableFee` · deployed as `StableFeeHarness`, a concrete stand-in for the target · 32 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_035_FablesStableFee` Stable builds: every swap must pay at least the pool's minimum fee (`setMinimumFee`, default 0 = no minimum) in both directions, inside and outside the band (`_getFee` raises lpFeeE12 to minimumFee*1e6); no discount may go below it (`_discounted` passes minimumFee as the floor of `DiscountLib.applyEntry`); lowering it must restart the falling fee (`_resetFeeState`) and emit `MinimumFeeSet`. `_resetFeeState` and the per-block state write must preserve the minimum-fee bits [240,256).
  check `check_inv_035_FablesStableFee` · deployed as `StableFeeHarness`, a concrete stand-in for the target · 38 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_038_FablesStableFee` Stable pool admin inputs: `configureStablePool` (once per pool, `_referenceOf(poolId) == 0`) validates config, resets fee state, writes cfg, sets `maxFee[poolId] = STABLE_MAX_FEE` and calls `poolManager.initialize`; `updateFeeConfig` rewrites k/optimalFeeE6/targetMultiplier and resets state; `updateReferencePrice` requires `block.timestamp >= referenceUpdatedAt + REFERENCE_UPDATE_INTERVAL` and `calculatePriceRatioX96(new, previous) >= MIN_REFERENCE_RATIO_X96`, then resets fee state and writes the reference; `setMinimumFee` writes state bits [240,256) and resets fee state only when lowering. `_resetFeeState` sets the decaying fee UNDEFINED, current clock and cached price 0, so the next swap reads a fresh price and restarts the decay from the far-boundary fee.
  check `check_inv_038_FablesStableFee` · deployed as `StableFeeHarness`, a concrete stand-in for the target · 1 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_059_FablesStableFee` `FablesStableFee._words[id].cfg` k / optimalFeeE6 / targetMultiplier / referenceSqrtPriceX96 and `FablesHookCore.maxFee[id]`: set by ADMIN via `FablesStableFee.configureStablePool` (once per pool: reverts if the reference is already non-zero; dynamic-fee key, hooks == this, native only on native builds; `k != 0` with no upper bound, `optimalFeeE6 <= 1e4`, `1 <= targetMultiplier <= 100`, reference within StablePair's bounded v4 range; initial `sqrtPriceX96` passed straight to `poolManager.initialize` with no link to the reference; `maxFee = STABLE_MAX_FEE`) and via `FablesStableFee.updateFeeConfig` (k, optimalFeeE6, targetMultiplier with the same bounds; reference kept; fee state reset). Read by `FablesStableFee._stableFee` -> `_getFee` / `_calculateDecayingFee` / `_targetFee` from the `FablesStableHookLite` fallback and `FablesStableHookLiteTyped.beforeSwap`, and by `getFee`, `currentFee`, `feeFor`, `feeConfig`.
  check `check_inv_059_FablesStableFee` · deployed as `StableFeeHarness`, a concrete stand-in for the target · 5 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_061_FablesStableFee` `FablesStableFee._entry[id][account]` and the 32-bit Bloom/seed in `FablesStableFee._words[id].cfg` bits [216,256): set by DISCOUNT_ADMIN via `FablesStableFee.setDiscount` (account != 0, bps <= 5000; for bps != 0: dirs in {1,2,3}, expiry in (now, now + 90 days]; pool initialized; minFee unbounded; bps == 0 deletes) and `FablesStableFee.setDiscountFilter` (any uint8 seed, Bloom rebuilt from listed accounts holding an entry); deleted by PAUSER via `FablesStableFee.revokeDiscount` (no pool check). Read by `FablesStableFee._stableFee` -> `FablesStableFee._discounted` when bloom != 0 and fee != 0: fee = min(fee, max(ceil(fee*(1 - min(bps,50%))), minimumFee, entry minFee)), and by `FablesStableFee.feeFor`.
  check `check_inv_061_FablesStableFee` · deployed as `StableFeeHarness`, a concrete stand-in for the target · 2 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_063_FablesStableFee` Shared state stable fee words `_words[poolId]`: `state` (decaying fee [0,40), cached sqrtPrice [40,200), clock block [200,240), minimum fee [240,256)) is written by the swap path `_stableFee` (only on the first swap of a clock block or after a reset; preserves min-fee bits), by `_resetFeeState` (from `configureStablePool`, `updateFeeConfig`, `updateReferencePrice`, and `setMinimumFee` when lowering) and by `setMinimumFee` (min-fee bits). `cfg` (k, optimalFeeE6, targetMultiplier, reference [0,216); bloom32/seed [216,256)) is written by `_writeFeeConfig` (keeps whitelist bits) and by the discount setters (whitelist bits).
  check `check_inv_063_FablesStableFee` · deployed as `StableFeeHarness`, a concrete stand-in for the target · 2 path(s) explored · sensitivity assessed: detected a deliberate change

FablesStableHookLite

- `inv_034_FablesStableHookLite` STABLE_KEEPER bounds: `updateReferencePrice` may move a pool's reference by about 0.1% at most per minute — requires block.timestamp >= referenceUpdatedAt + REFERENCE_UPDATE_INTERVAL (60 s), `_validateReferenceSqrtPriceX96`, and calculatePriceRatioX96(new, previous) >= MIN_REFERENCE_RATIO_X96 (= Q96 * 999_000 / 1e6, symmetric in direction); it resets the fee state and sets referenceUpdatedAt = now. `setMinimumFee` takes a uint16 (at most 65,535 pips), has no rate limit, and resets the fee state only when lowering. STABLE_KEEPER must not move funds or change any other setting.
  check `check_inv_034_FablesStableHookLite` · deployed as `StableHookHarness`, a concrete stand-in for the target · 9 path(s) explored · sensitivity unassessed
- `inv_060_FablesStableHookLite` Stable reference price (`referenceSqrtPriceX96` in `FablesStableFee._words[id].cfg`) and `FablesStableHookLite.referenceUpdatedAt[id]`: moved by STABLE_KEEPER via `FablesStableHookLite.updateReferencePrice`, bounded to pool initialized, `block.timestamp >= referenceUpdatedAt + REFERENCE_UPDATE_INTERVAL` (60 s), new reference within the validated range, and `calculatePriceRatioX96(new, previous) >= MIN_REFERENCE_RATIO_X96` (0.999 in Q96); k/optimalFeeE6/targetMultiplier and the whitelist bits kept; fee state reset (decaying fee UNDEFINED, cached price 0). No cumulative bound across calls. Read by `FablesStableFee._getFee` (priceRatio = ratio of AMM sqrt price to reference; picks inside-band fee vs close/far boundary fees and which side pays 0), `_decayStartFee`, `_targetFee` (floor beyond ratio 0.9), and views `getFee` / `currentFee` / `feeFor` / `feeConfig`.
  check `check_inv_060_FablesStableHookLite` · 2 path(s) explored · sensitivity unassessed
- `inv_071_FablesStableHookLite` Each updateReferencePrice requires ratio >= 0.999 and setMinimumFee takes a uint16 (at most 65,535 pips).
  check `check_inv_071_FablesStableHookLite` · 1 path(s) explored · sensitivity unassessed

FablesFeeFull

- `inv_006_FablesFeeFull` For the same state and router, the fee a swap pays must equal what `feeFor` quotes; `currentFee` quotes it without a discount. Fee-engine builds: the assembly `fallback` in `FablesFeeFull` must match `FeeRef.resolve` / `_resolveFeeRef` bit for bit (keeper fee, discount for the PoolManager.swap caller, CF surcharge with the CF-word update, cap). Stable builds: the assembly `fallback` around `_stableFee` (and `FablesStableHookLiteTyped.beforeSwap`) must match `getFee`, `currentFee` and `feeFor(key, zeroForOne, router)`, including the target floor, the 10% cap, the minimum fee and the discount.
  check `check_inv_006_FablesFeeFull` · deployed as `FablesHook`, a concrete stand-in for the target · 33 path(s) explored · sensitivity unassessed
- `inv_028_FablesFeeFull` FablesHook swap fee (`FablesFeeFull` fallback, mirrored by `FeeRef.resolve` / `_resolveFeeRef`): from the pool word `_word[id]`, `fee = base[side]`; if `block.timestamp < expiry` and `poke[side] != 0`, `fee = max(poke[side], max(floor, min(base[side], cap) >> 1))`; `fee = min(fee, cap)`; if `bloom != 0` and the PoolManager.swap `sender` hits the bloom and `_entry[id][sender]` is live for this direction, `fee = min(fee, max(ceil(fee*(10000-min(bps,5000))/10000), floor, minFee))`; if `cfOn`, `fee = min(fee + s, cap)` with `s` from the CF step. Returned as `fee | OVERRIDE_FEE_FLAG`. Inputs: base/floor/cap/category by ADMIN `configurePool`; pokes/expiry by FEE_POKER `pokeFee`/`clearPoke`; entries/bloom by DISCOUNT_ADMIN `setDiscount`/`setDiscountFilter`/`revokeDiscount`; `sender` is whichever contract calls `PoolManager.swap` (any user-chosen router); CF inputs move with the pool tick (any swapper) and time.
  check `check_inv_028_FablesFeeFull` · deployed as `FablesHook`, a concrete stand-in for the target · 1950 path(s) explored · sensitivity unassessed

DiscountLib

- `inv_011_DiscountLib` Discounts for listed senders never raise a fee or take it below the floor.
  check `check_inv_011_DiscountLib` · deployed as `DiscountHarness`, a concrete stand-in for the target · 52 path(s) explored · sensitivity assessed: detected a deliberate change
- `inv_012_DiscountLib` A DISCOUNT_ADMIN discount must never raise any fee and never take it below the floor (fee-engine builds: the pool `floor`; stable builds: the pool's minimum fee). The discounted fee must be <= the undiscounted fee and >= min(undiscounted fee, floor).
  check `check_inv_012_DiscountLib` · deployed as `DiscountLibHarness`, a concrete stand-in for the target · 52 path(s) explored · sensitivity assessed: detected a deliberate change

FablesHookCore

- `inv_064_FablesHookCore` `AccessManaged.authority` of every hook build (set in the `FablesHookCore` constructor): replaceable only by the current authority via `AccessManaged.setAuthority` (AccessManager ADMIN through `updateAuthority`), with no bound on the new address; role-to-selector bindings, role grants and execution/admin delays live in the AccessManager and are changed by ADMIN. Read by the `restricted` modifier on every setter of `FablesFeeEngine`, `FablesLedgerV2`, `FablesStableFee` and `FablesStableHookLite`. The hooks have no proxy or upgrade path; `poolManager`, `weth` and `USE_ARB_SYS` are immutables.
  check `check_inv_064_FablesHookCore` · deployed as `FablesHookCoreHarness`, a concrete stand-in for the target · 1 path(s) explored · sensitivity unassessed
- `inv_068_FablesHookCore` `FablesHookCore.maxFee[id]` is written only by `FablesFeeEngine.configurePool` (= cap, cap >= 1) and `FablesStableFee.configureStablePool` (= STABLE_MAX_FEE), never to 0; `FablesLedgerV2._checkKey` and `FablesFeeEngine.initializePool` treat a non-zero value as "configured". The core NatSpec states the marker is never reset to 0.
  check `check_inv_068_FablesHookCore` · deployed as `FablesHookCoreHarness`, a concrete stand-in for the target · 10 path(s) explored · sensitivity unassessed

AccWrapHarness

- `inv_077_AccWrapHarness` `_ranges[ID].totalShares` equals the sum of `balanceOf` over all accounts; `join` and `leave` change both by the same amount.
  check `check_inv_077_AccWrapHarness` · 11 path(s) explored · sensitivity unassessed
- `inv_078_AccWrapHarness` `_ranges[ID].totalStaked` equals the sum of `_pos[who].staked` over all accounts, and `_ranges[ID].totalStaked` is at most `_ranges[ID].totalShares`. Each account's staked amount is at most its `balanceOf`. `unstake` does not check the amount against the staked balance, so it relies on checked arithmetic.
  check `check_inv_078_AccWrapHarness` · 11 path(s) explored · sensitivity unassessed

FeeRef

- `inv_009_FeeRef` The CF anchor updates once per period; if no swap updates it for min(255 s, 5 x tau) it snaps to the price.
  check `check_inv_009_FeeRef` · deployed as `FeeRefHarness`, a concrete stand-in for the target · 8 path(s) explored · sensitivity assessed: detected a deliberate change

LedgerHandler

- `inv_073_LedgerHandler` `LedgerHandler.callCount` is a cumulative counter that only increases, by exactly 1 per handler call, and is never decreased or reset by any handler function.
  check `check_inv_073_LedgerHandler` · 3 path(s) explored · sensitivity unassessed

ConservationHandler

- `inv_074_ConservationHandler` `ConservationHandler.callCount` and `ConservationHandler.slackGrowths` only increase, by at most 1 per handler call.
  check `check_inv_074_ConservationHandler` · 3 path(s) explored · sensitivity unassessed

IsolationHandler

- `inv_075_IsolationHandler` `pausesLanded` is a cumulative counter that only increases (by one per successful `pausePool` or `liftPool`).
  check `check_inv_075_IsolationHandler` · 6 path(s) explored · sensitivity unassessed

EngineInvariantHandler

- `inv_076_EngineInvariantHandler` `fieldViolations` is a cumulative counter that only increases (via ++) and stays 0 if the hook preserves non-poke fields of the pool word.
  check `check_inv_076_EngineInvariantHandler` · 2 path(s) explored · sensitivity unassessed

Findings — violated invariants

Finding · `inv_004_FablesFeeEngine` — FablesFeeEngine

ID	`inv_004_FablesFeeEngine`
Contract	FablesFeeEngine
Criticality	High
Verdict	Violated
Prover time	25.8s

Invariant. Only the hook can initialize its pools; on the Exclusive builds only the ledger can add liquidity.

Impact if violated. Outsiders initialize hook pools at arbitrary prices or with unconfigured fees, or add unreceipted/JIT liquidity on Exclusive builds that bypasses the ledger, claim fee and pause.

PROVER COUNTEREXAMPLE — VERBATIM OUTPUT

```
Counterexample:
    halmos_flagbits_uint256_d875235_01 = 0x04
```

Reproduce with `forge test --match-path inv_004_FablesFeeEngine.t.sol`

Finding · `inv_036_FablesFeeEngine` — FablesFeeEngine

ID	`inv_036_FablesFeeEngine`
Contract	FablesFeeEngine
Criticality	Medium
Verdict	Violated
Prover time	75.5s

Invariant. `FablesFeeEngine._categories[id]` (floorMin, capMax, raiseStep): set by ADMIN via `FablesFeeEngine.setCategory`, bounded to `capMax <= ABSOLUTE_MAX_FEE` (100_000 pips), `floorMin <= capMax`, `raiseStep != 0`; overwrites an existing category without touching pools already in it. Read by `FablesFeeEngine.configurePool` (floor >= floorMin, cap <= capMax at configure time) and `FablesFeeEngine.pokeFee` (poke ceiling = min(pool cap, current capMax); max rise per raising poke = current raiseStep). Not read by the swap entry (`FablesFeeFull` fallback), which clamps only to the pool's stored cap, so lowering capMax leaves existing pool caps and live pokes in force.

Impact if violated. A loose category lets ADMIN configure caps up to 10% and lets FEE_POKER raise fees in large steps; tightening a category does not retroactively bind existing pool caps on the swap path.

PROVER COUNTEREXAMPLE — VERBATIM OUTPUT

```
Counterexample:
    halmos_x_uint256_accb940_01 = 0x00
```

Reproduce with `forge test --match-path inv_036_FablesFeeEngine.t.sol`

Finding · `inv_005_FablesFeeEngine` — FablesFeeEngine

ID	`inv_005_FablesFeeEngine`
Contract	FablesFeeEngine
Criticality	High
Verdict	Violated
Prover time	22.9s

Invariant. The LP fee a swap pays (the fee returned with OVERRIDE_FEE_FLAG by every build's beforeSwap entry) must be <= the pool cap and <= 100,000 pips (10%). Fee-engine builds: `FeeRef.keeperFee` clamps to `w.cap()` and `FeeRef.resolve` clamps again after the CF surcharge; `configurePool` requires `cap <= capMax` and `setCategory` requires `capMax <= ABSOLUTE_MAX_FEE` (100_000). Stable builds: `_toPips` caps at `STABLE_MAX_FEE` (100_000) and `configureStablePool` writes `maxFee[id] = STABLE_MAX_FEE`; the minimum fee (uint16, <= 65,535 pips) and discount must not push the charged fee above the cap.

Impact if violated. Swappers overcharged beyond the documented 10% hard ceiling or beyond the pool cap; breaks the ADMIN 'cannot set a fee above 10%' guarantee.

PROVER COUNTEREXAMPLE — VERBATIM OUTPUT

```
Counterexample:
    halmos_createdAddr_uint256_89ede72_01 = 0xaaaa0002
```

Reproduce with `forge test --match-path inv_005_FablesFeeEngine.t.sol`

Finding · `inv_008_FablesFeeEngine` — FablesFeeEngine

ID	`inv_008_FablesFeeEngine`
Contract	FablesFeeEngine
Criticality	High
Verdict	Violated
Prover time	20.9s

Invariant. FEE_POKER can turn the continuation fee on or re-anchor it, only at a tick inside the range the call gives and by its deadline, or off.

Impact if violated. A semi-trusted keeper overcharges or undercharges swappers beyond documented limits.

PROVER COUNTEREXAMPLE — VERBATIM OUTPUT

```
Counterexample:
    halmos_engineFlagBits_uint256_0d4693d_01 = 0x04
```

Reproduce with `forge test --match-path inv_008_FablesFeeEngine.t.sol`

Inconclusive results

1 invariant returned **inconclusive** — the prover exhausted its per-invariant time budget or hit an unsupported construct before reaching either a proof or a counterexample. An inconclusive verdict is not a finding: it marks an invariant whose proof needs a larger solver budget, a simplified formulation, or a manual argument. Re-running with a longer timeout typically resolves the majority of these.

07 Deliverables & next steps

What is in this package

This PDF report, plus `invariant_tests.zip` — the generated symbolic Foundry test behind every translated invariant: 62 proof(s) and 4 counterexample reproduction(s), each runnable in your own repository.

How to re-run a verdict

Unpack the test bundle into your Foundry workspace and run any test with `forge test --match-path <file>`. Proofs re-verify; violated tests reproduce their counterexample.

Included test files

- `inv_001_FablesLedgerV2.t.sol`
- `inv_002_FablesLedgerV2.t.sol`
- `inv_003_FablesLedgerV2.t.sol`
- `inv_004_FablesFeeEngine.t.sol`
- `inv_006_FablesFeeFull.t.sol`
- `inv_007_FablesFeeEngine.t.sol`
- `inv_009_FeeRef.t.sol`
- `inv_011_DiscountLib.t.sol`

Inconclusive invariants ship with their generated tests too — re-running them with a longer prover budget is the fastest path to raising the proven count.

A Appendix — full invariant table

Every invariant in scope, its verdict, and the prover effort spent on it.

ID	Contract	Crit.	Type	Verdict	Time	Invariant statement	Impact if viola
`inv_001_FablesLedgerV2`	FablesLedgerV2	High	behavioural	Check passed	41.1s	`withdraw` must succeed for a holder of receipts regardless of any role action, pause or configuration. Code: `withdraw` has no `whenNotPaused`/`whenPoolNotPaused` modifier and no role gate; `_withdrawTo` syncs the range (`_syncRange`), settles the user (`settleUser`, with saturating `_accrue` and unchecked `forgone` sums), auto-unstakes any shortfall from staked shares, burns receipts and removes liquidity through `_modify`; fees are credited, not paid. Admin-settable state it touches includes the claim-fee history walk (`claimFeeFor`), claim-fee recipient, distributor/sinks and the ballot gate; none of it may make `withdraw` revert. `withdrawAndClaim` is pausable and is not the guaranteed exit.	LP principal loc sink/gate confi revert in the sy from exiting.
`inv_002_FablesLedgerV2`	FablesLedgerV2	High	conservation:le	Check passed	10.0s	The ledger never owes more than it holds.	Insolvency: late withdrawers ca drains fees or p
`inv_003_FablesLedgerV2`	FablesLedgerV2	High	temporal_ordering	Check passed	81.2s	Fees earned before a deposit, stake or transfer must not be credited to the newcomer. Mechanism: every share/stake change (`deposit`, `_withdrawTo`, `stake`, `unstake`, `transfer`, `transferFrom`) first calls `_syncRange(id)` (collects v4 fees at current totals via `creditFees`/`_split`) and then `_settleUser` for each affected account, which sets the account's `checkpoint*X128`/`stakedCheckpoint*X128` to the current accumulator levels before the balance changes. A recreated range keeps old accumulator levels; the new depositor's checkpoint is set to them.	JIT-style fee the LPs/stakers by receiving recei
`inv_004_FablesFeeEngine`	FablesFeeEngine	High	authorization	Violated	25.8s	Only the hook can initialize its pools; on the Exclusive builds only the ledger can add liquidity.	Outsiders initia arbitrary prices fees, or add un Exclusive build ledger, claim fe
`inv_006_FablesFeeFull`	FablesFeeFull	High	behavioural	Check passed	10.9s	For the same state and router, the fee a swap pays must equal what `feeFor` quotes; `currentFee` quotes it without a discount. Fee-engine builds: the assembly `fallback` in `FablesFeeFull` must match `FeeRef.resolve` / `_resolveFeeRef` bit for bit (keeper fee, discount for the PoolManager.swap caller, CF surcharge with the CF-word update, cap). Stable builds: the assembly `fallback` around `_stableFee` (and `FablesStableHookLiteTyped.beforeSwap`) must match `getFee`, `currentFee` and `feeFor(key, zeroForOne, router)`, including the target floor, the 10% cap, the minimum fee and the discount.	Quotes diverge swappers overp the assembly p (cap, floor, disc enforced in the
`inv_007_FablesFeeEngine`	FablesFeeEngine	High	formula	Check passed	866.7s	Pokes move each side's fee within [max(floor, 50% of base), min(pool cap, category cap)] for at most 72 h, raising at most one step per 60 s.	FEE_POKER car documented ba step per 60 s, c 72 h.
`inv_009_FeeRef`	FeeRef	Medium	formula	Check passed	128.9s	The CF anchor updates once per period; if no swap updates it for min(255 s, 5 x tau) it snaps to the price.	Surcharge exce cap, or the anc charge one dire
`inv_011_DiscountLib`	DiscountLib	Medium	formula	Check passed	7.0s	Discounts for listed senders never raise a fee or take it below the floor.	Discount raises the pool floor / to unlisted sen
`inv_012_DiscountLib`	DiscountLib	Medium	bound:le	Check passed	6.9s	A DISCOUNT_ADMIN discount must never raise any fee and never take it below the floor (fee-engine builds: the pool `floor`; stable builds: the pool's minimum fee). The discounted fee must be <= the undiscounted fee and >= min(undiscounted fee, floor).	DISCOUNT_ADM listed routers o drain LP fee rev
`inv_013_FablesLedgerV2`	FablesLedgerV2	High	bound:le	Check passed	8.1s	PAUSER can pause deposits, claims, stakes, transfers and revenue sweeps for up to 7 days per call, clear pokes, switch off the continuation fee and revoke discounts.	Pauser locks LP ledger beyond
`inv_014_FablesFeeEngine`	FablesFeeEngine	High	authorization	Check passed	1329.4s	ADMIN powers include setting the claim fee, but the quote only names moving LP principal, setting a fee above 10% and un-configuring a pool as listed limits.	Admin steals pr 10%, or un-con ledger operatio
`inv_015_FablesLedgerV2`	FablesLedgerV2	High	behavioural	Check passed	10.7s	LP principal entry: `FablesLedgerV2.deposit(key, tickLower, tickUpper, liquidity, amount0Max,	Receipts minte principal, or pri

ID	Contract	Crit.	Type	Verdict	Time	Invariant statement	Impact if viola
						amount1Max, deadline)` registers `_rangeKeys[id]` (id = keccak256(abi.encode(poolId, tickLower, tickUpper))) on first use, calls `_syncRange(id)` and `_settleUser(id, msg.sender)`, increments `_ranges[id].resetSeq` if `totalShares` was 0, raises `_ranges[id].totalShares` by `liquidity`, mints `liquidity` ERC-6909 receipts to `msg.sender`, then `_modify` opens a PoolManager unlock in which `LedgerLibV2._onModify` calls `modifyLiquidity(+liquidity, salt 0)` on the hook-owned position and `_settlePrincipal` pulls `principal = callerDelta - feesAccrued` per currency: ERC20 via `safeTransferFrom(msg.sender, poolManager, owed)` + `settle()`, native via `settle{value: owed}()` from the hook's balance. `_checkBound` reverts if paid principal exceeds `amount0Max`/`amount1Max`. On native pools `_refundNative` sends `address(this).balance - preBalance` back to `msg.sender` last.	wrong party / n bound; refund p to others.
`inv_016_FablesLedgerV2`	FablesLedgerV2	High	behavioural	Check passed	8.2s	Withdrawing and unstaking are never pausable or gated.	Principal paid fo mismatched, st without updatir blocked.
`inv_017_LedgerLibV2`	LedgerLibV2	High	formula	Check passed	11.6s	Fee collection and split (`LedgerLibV2.creditFees` → `_split`): `_syncRange(id)` (skipped when `_rangeKeys[id].set` is false or `totalShares == 0`, which only rolls `claimFeeSyncIndex[id]`) runs `modifyLiquidity(0)` on the range; `feesAccrued` (fee0, fee1) are minted as PoolManager ERC-6909 claims to the hook. With `total = range.totalShares`: if `total == 0` all fees go to `distributorOwed`; else `stakedCut = _takeStakedCut(fee, total)` goes to `distributorOwed` and raises `accStaked*X128 += cut * 2^128 / totalStaked`; if `unstakedTotal = total - totalStaked == 0` the remainder also goes to `distributorOwed`; otherwise `net = fee - stakedCut`, claim fee `c = net * feeBps / 10_000` (only when `feeBps != 0` and `claimFeeRecipient != 0`) goes to `claimFeeOwed`, and `accFee*X128 += (net - c) * 2^128 / unstakedTotal` (unchecked, wraps). Inputs: fee0/fee1 grow with any swap in the pool (any user), `totalShares`/`totalStaked` move via deposit/withdraw/stake/unstake (any holder), `feeBps` via REVENUE_OPS/ADMIN setters.	Fees credited a depositor/stake before they joir to the distribute vice versa).
`inv_018_LedgerLibV2`	LedgerLibV2	Medium	formula	Check passed	47.7s	Staked cut with carried remainder: `LedgerLibV2._takeStakedCut` computes `carried = stakedResidual + mulDiv(mulmod(fee, totalStaked, total), 2^128, total)` and `cut = mulDiv(fee, totalStaked, total) + (carried >> 128)`, storing `uint128(carried)` back into `range.stakedResidual0/1`. The unstaked side floors twice (`_split` comment: dust under one unit per collection stays as hook-held claims). `totalStaked` and `total` are moved by any holder through `stake`, `unstake`, `deposit`, `withdraw` before the next sync.	Rounding drift l unstaked sides; over-flushed ch revenue.
`inv_019_LedgerLibV2`	LedgerLibV2	High	formula	Check passed	48.6s	settleUser is called before every change to the holder's share count.	A balance or st settlement crec accrued on sha loses fees they
`inv_020_FablesLedgerV2`	FablesLedgerV2	High	bound:le	Check passed	873.3s	The claim fee can be set, at most 20% of LP fees.	Revenue role ta fees or skims p
`inv_021_LedgerLibV2`	LedgerLibV2	High	formula	Check passed	33.3s	The claim fee is charged at no more than the lowest rate in force since a range's last sync, and its recipient is hook-wide.	LPs charged ab force since last staked/principa
`inv_022_LedgerLibV2`	LedgerLibV2	Medium	documented_choice:both	Check passed	19.8s	The claim fee is charged at no more than the lowest rate in force since a range's last sync.	If the code cha rate since last s choice itself lov
`inv_023_LedgerLibV2`	LedgerLibV2	High	formula	Check passed	53.5s	Rounding favours the pool: fee splits never credit more than was collected, and holder accruals round down.	Over-crediting (fees between s and sinks, or pe donations into s
`inv_024_LedgerLibV2`	LedgerLibV2	Medium	documented_choice:both	Check passed	46.5s	The staked cut carries its remainder to the next collection, and the claim fee rounds down in LPs' favour.	Rounding in the accumulates in
`inv_025_LedgerLibV2`	LedgerLibV2	Medium	formula	Check passed	29.4s	A claim-fee cut applies to the whole backlog; a raise only to later ones.	A raise applied accrued under cut/raise mis-pı or history walk
`inv_026_LedgerLibV2`	LedgerLibV2	Medium	bound:le	Check passed	33.4s	Claim fee is bounded at 20% of the unstaked LP-fee residue: `LedgerLibV2.recordClaimFee` reverts	REVENUE_OPS of LP fees.

ID	Contract	Crit.	Type	Verdict	Time	Invariant statement	Impact if viola
						when `bps > MAX_CLAIM_FEE_BPS (2000)`, and `_split` applies `feeBps` only to `net = fee - stakedCut`.	
`inv_027_FablesLedgerV2`	FablesLedgerV2	High	behavioural	Check passed	19.8s	Holder fee payout: `claimFees(key, tickLower, tickUpper, to, maxFeeBps)` (whenPoolNotPaused) reverts `NothingToClaim` when the caller has zero balance and zero owed, runs `_syncRange` + `_settleUser`, zeroes `position.owed0/owed1`, then `_payout` opens a PAYOUT unlock where `LedgerLibV2._redeem` does `poolManager.burn(hook, currency.toId(), amount)` and `poolManager.take(currency, to, amount)` for each currency. `withdrawAndClaim` does the same after `_withdrawTo`. The hook's PoolManager claim balance (minted in `creditFees`) is the source of every payout.	Owed amounts claim balance r revert; double p zeroed.
`inv_028_FablesFeeFull`	FablesFeeFull	High	formula	Check passed	954.8s	FablesHook swap fee (`FablesFeeFull` fallback, mirrored by `FeeRef.resolve` / `_resolveFeeRef`): from the pool word `_word[id]`, `fee = base[side]`; if `block.timestamp < expiry` and `poke[side] != 0`, `fee = max(poke[side], max(floor, min(base[side], cap) >> 1))`; `fee = min(fee, cap)`; if `bloom != 0` and the PoolManager.swap `sender` hits the bloom and `_entry[id][sender]` is live for this direction, `fee = min(fee, max(ceil(fee* (10000-min(bps,5000))/10000), floor, minFee))`; if `cfOn`, `fee = min(fee + s, cap)` with `s` from the CF step. Returned as `fee | OVERRIDE_FEE_FLAG`. Inputs: base/floor/cap/category by ADMIN `configurePool`; pokes/expiry by FEE_POKER `pokeFee`/`clearPoke`; entries/bloom by DISCOUNT_ADMIN `setDiscount`/`setDiscountFilter`/`revokeDiscount`; `sender` is whichever contract calls `PoolManager.swap` (any user-chosen router); CF inputs move with the pool tick (any swapper) and time.	Fee above the p floor, discounts senders, or ass making quotes
`inv_029_FablesFeeEngine`	FablesFeeEngine	Medium	bound:both	Check passed	104.8s	Pokes move each side's fee within [max(floor, 50% of base), min(pool cap, category cap)] for at most 72 h, raising at most one step per 60 s.	FEE_POKER mo documented ba one step per 60
`inv_030_FablesFeeEngine`	FablesFeeEngine	Medium	bound:le	Check passed	16.7s	Pool fee configuration bounds: `setCategory` requires `capMax <= ABSOLUTE_MAX_FEE (100_000 = 10%)`, `floorMin <= capMax`, `raiseStep != 0`; `configurePool` requires dynamic-fee key with `hooks == this`, an existing category, `1 <= cap <= capMax`, `max(base0, base1) <= cap`, `floorMin <= floor <= min(base0, base1)`, writes bases/floor/cap into `_word[id]` and `maxFee[id] = cap` (kept pokes, CF and filter). Existing pools keep stored bounds when a category is later changed. ADMIN may move bases anywhere inside [floor, cap] with no raiseStep limit.	A pool fee abov configured (ma ledger operatio
`inv_031_FablesStableFee`	FablesStableFee	High	formula	Check passed	18.6s	Swaps back pay a fee that starts high and falls every block, and the fee is capped at 10%.	Corrective (bac swaps underpri extract LP valu overcharged.
`inv_032_FablesStableFee`	FablesStableFee	High	bound:ge	Check passed	869.3s	Stable builds: the charged fee must be >= StablePair's fee for the same state capped at 10% (i.e. >= min(StablePair fee, 100_000 pips)) — never below it, possibly above it. Once the price ratio to the reference is below 0.9, only the decay target is floored at its value at 0.9 (`_targetFee`); the formula must read and store the real (unclamped) AMM price (as StablePair does) so the movement adjustment in `_decayStartFee` uses real prices; the floor only ever raises the target. (Fix for pre-audit findings 5.1.1 a/b.)	After a depeg n arbitrageurs sw than StablePair LP value.
`inv_034_FablesStableHookLite`	FablesStableHookLite	Medium	bound:le	Check passed	155.3s	STABLE_KEEPER bounds: `updateReferencePrice` may move a pool's reference by about 0.1% at most per minute — requires block.timestamp >= referenceUpdatedAt + REFERENCE_UPDATE_INTERVAL (60 s), `_validateReferenceSqrtPriceX96`, and calculatePriceRatioX96(new, previous) >= MIN_REFERENCE_RATIO_X96 (= Q96 * 999_000 / 1e6, symmetric in direction); it resets the fee state and sets referenceUpdatedAt = now. `setMinimumFee` takes a uint16 (at most 65,535 pips), has no rate limit, and resets the fee state only when lowering. STABLE_KEEPER must not move funds or change any other setting.	Keeper moves documented, m charge correcti fee direction.
`inv_035_FablesStableFee`	FablesStableFee	Medium	bound:ge	Check passed	185.9s	Stable builds: every swap must pay at least the pool's minimum fee (`setMinimumFee`, default 0 = no minimum) in both directions, inside and outside	Swaps (especia direction ones) minimum fee.

ID	Contract	Crit.	Type	Verdict	Time	Invariant statement	Impact if viola
						the band (`_getFee` raises lpFeeE12 to minimumFee*1e6); no discount may go below it (`_discounted` passes minimumFee as the floor of `DiscountLib.applyEntry`); lowering it must restart the falling fee (`_resetFeeState`) and emit `MinimumFeeSet`. `_resetFeeState` and the per-block state write must preserve the minimum-fee bits [240,256).	
`inv_036_FablesFeeEngine`	FablesFeeEngine	Medium	formula	Violated	75.5s	`FablesFeeEngine._categories[id]` (floorMin, capMax, raiseStep): set by ADMIN via `FablesFeeEngine.setCategory`, bounded to `capMax <= ABSOLUTE_MAX_FEE` (100_000 pips), `floorMin <= capMax`, `raiseStep != 0`; overwrites an existing category without touching pools already in it. Read by `FablesFeeEngine.configurePool` (floor >= floorMin, cap <= capMax at configure time) and `FablesFeeEngine.pokeFee` (poke ceiling = min(pool cap, current capMax); max rise per raising poke = current raiseStep). Not read by the swap entry (`FablesFeeFull` fallback), which clamps only to the pool's stored cap, so lowering capMax leaves existing pool caps and live pokes in force.	A loose categor caps up to 10% raise fees in lar category does existing pool ca
`inv_037_FablesFeeEngine`	FablesFeeEngine	High	formula	Check passed	12.6s	`FablesFeeEngine._word[id]` bases/floor/cap, `FablesFeeEngine._meta[id].category` and `FablesHookCore.maxFee[id]`: set by ADMIN via `FablesFeeEngine.configurePool`, bounded to `key.hooks == this`, dynamic-fee key, existing category, `c.floorMin <= floor <= min(base0, base1)`, `max(base0, base1) <= cap <= c.capMax`, `cap >= 1`; `maxFee[id] = cap`. Callable again on an initialized pool (reconfigure keeps pokes, Bloom filter and CF state). Read by the `FablesFeeFull` fallback (beforeSwap fee), `FablesFeeFull.currentFee`, `FablesFeeFull.feeFor`, `FablesFeeEngine.pokeFee` (floor/cap bounds), discount floor; `maxFee` read by `FablesLedgerV2._checkKey` (deposit, withdraw, withdrawAndClaim, claimFees, stake, setClaimFee, setPoolPaused, pausePool) and `FablesFeeEngine.initializePool`.	Bases and cap fee (up to capM reconfigure on immediately fo
`inv_038_FablesStableFee`	FablesStableFee	Medium	behavioural	Check passed	7.4s	Stable pool admin inputs: `configureStablePool` (once per pool, `_referenceOf(poolId) == 0`) validates config, resets fee state, writes cfg, sets `maxFee[poolId] = STABLE_MAX_FEE` and calls `poolManager.initialize`; `updateFeeConfig` rewrites k/optimalFeeE6/targetMultiplier and resets state; `updateReferencePrice` requires `block.timestamp >= referenceUpdatedAt + REFERENCE_UPDATE_INTERVAL` and `calculatePriceRatioX96(new, previous) >= MIN_REFERENCE_RATIO_X96`, then resets fee state and writes the reference; `setMinimumFee` writes state bits [240,256) and resets fee state only when lowering. `_resetFeeState` sets the decaying fee UNDEFINED, current clock and cached price 0, so the next swap reads a fresh price and restarts the decay from the far-boundary fee.	Reference mov step bound, or swaps changes swapper.
`inv_039_FablesFeeEngine`	FablesFeeEngine	High	formula	Check passed	7.8s	`FablesFeeEngine._cf[id]` (mode, kQ8, cfCap, band16, tau/decay1, period, maxAge, anchor, tLast) and the `cfOn` bit of `FablesFeeEngine._word[id]`: parameters set by ADMIN via `FablesFeeEngine.configureCF`, bounded to pool configured, CF off, `mode <= 1`, `1 <= kQ8 <= 256`, `1 <= cfCap <= 2500`, `band16 <= 800`, `5 <= tau <= 300`, `1 <= period <= 60`, `period < min(255, 5*tau)`; anchor = current tick * 256. Switched on (re-anchored at current tick in [minTick, maxTick], before deadline, kQ8 != 0) or off by FEE_POKER via `FablesFeeEngine.setCFActive`; killed (off, kQ8 = 0) by PAUSER via `FablesFeeEngine.killCF`. Read by the `FablesFeeFull` fallback when cfOn: anchor A decays toward T = tick*256 by decay1^dt when dt >= period (snap at maxAge) and is written back; x = (zeroForOne ? A - T : T - A), negated in mode 1, minus band16*16; if x > 0, fee = min(fee + min((kQ8 * x * 100) >> 16, cfCap), cap).	The anchor tick which swaps pa anchor is set fr which swappers configureCF/set by setCFActive deadline).
`inv_041_FablesFeeEngine`	FablesFeeEngine	Medium	behavioural	Check passed	7.9s	Every role setter writes the values its caller signs (a fee, a duration, a reference, a minimum) and re-checks all of its bounds when it executes (e.g. `pokeFee` ttl/expiry from block.timestamp and raise interval, `setPaused` duration, `updateReferencePrice` step vs current reference and interval, `setMinimumFee` value, `setDiscount` expiry vs now), so a late-executed signed call changes only when those values apply,	A delayed or re applies values picks a price-de whoever lands

ID	Contract	Crit.	Type	Verdict	Time	Invariant statement	Impact if viola
						never what they are or how far they can go. The one price-dependent role call, `setCFActive(true)`, takes tick bounds [minTick, maxTick] and a deadline. Role setters must not derive values from manipulable live state beyond these bounds.	
`inv_042_FablesLedgerV2`	FablesLedgerV2	Medium	behavioural	Check passed	8.6s	A transfer of 0 reverts; staked receipts must be unstaked before they move.	Staked receipts fees to the dist misattributed o
`inv_043_FablesLedgerV2`	FablesLedgerV2	Medium	behavioural	Check passed	13.7s	Sink changes: `setClaimFee`, `setDistributor` and `setFFFeeSink` first call `_settleSinks(settleFirst)`, paying the current sinks the booked pots (`distributorOwed` / `claimFeeOwed`) of the listed currencies; everything else, including fees not yet synced into the ledger, is paid to the sinks set when it is swept. Sweeps (`sweepDistributor`, `sweepClaimFee`) are permissionless, pay `distributorOwed[currency]` to `ffFeeSink` when currency == `distributorFF` else to `distributor`, and `claimFeeOwed` to `claimFeeRecipient`; a sweep to a zero target is a no-op keeping the accrual. Native revenue is paid in WETH. The claim-fee recipient is hook-wide (any pool's `setClaimFee` rewrites it for all pools) and never returns to zero once set.	Revenue lost, d wrong sink acro
`inv_044_FablesLedgerV2`	FablesLedgerV2	High	behavioural	Check passed	8.7s	Shared state `_ranges[id].totalShares`: raised by `deposit` (+liquidity, after `_syncRange`/`_settleUser`), lowered by `_withdrawTo` (`withdraw`, `withdrawAndClaim`, −liquidity); `resetSeq` is incremented when `totalShares` goes 0→positive (deposit) and when it reaches 0 (withdraw). It is the denominator of the staked cut in `_split` and the `total == 0` branch sends all fees to `distributorOwed`. It always moves together with ERC-6909 `_mint`/`_burn` of the same amount; `transfer`/`transferFrom` move `balanceOf` without changing it.	Divergence bet sum of receipt shares and can
`inv_045_FablesLedgerV2`	FablesLedgerV2	High	behavioural	Check passed	10.7s	Shared state `distributorOwed[currency]` and `claimFeeOwed[currency]` (hook-wide per currency, summed across all pools/ranges): raised only inside `LedgerLibV2._split` (from `creditFees`, reached by every `_syncRange`/`_modify` path: deposit, withdraw, withdrawAndClaim, claimFees, stake, unstake, transfer, transferFrom, syncRanges); reset to 0 by `sweepDistributor`/`sweepClaimFee` (anyone, when not paused hook-wide and sink set) and by `_settleSinks` inside `setClaimFee`, `setDistributor`, `setFFFeeSink`. Holder `position.owed0/owed1` is raised by `settleUser` and reset to 0 by `claimFees` and `withdrawAndClaim`. All three are paid from the same pool of hook-held PoolManager claims per currency.	If the sum of di claimFeeOwed hook's PoolMan currency, the la cannot be paid.
`inv_046_FablesLedgerV2`	FablesLedgerV2	Medium	behavioural	Check passed	9.6s	Shared state claim-fee policy: `claimFeeRecipient` is hook-wide while `_poolClaimFee[poolId]` (bps, history, allTimeMin) is per pool. `setClaimFee` (sweeps `settleFirst`, then pushes a history entry `{fromTs, effective bps}` for one pool and overwrites the hook-wide `claimFeeRecipient`; reverts when trying to set recipient back to 0 once set) and `setClaimFeeBps` (requires a recipient, pushes an entry for one pool, no sweep) are the only writers; `recordClaimFee` rejects recipient == hook, == `distributor`, == `ffFeeSink`. `claimFeeSyncIndex[id]` is written by `creditFees` (every non-idle sync) and by `_syncRange`'s idle branch, always to the pool's current history length.	A recipient cha setClaimFee re revenue; histor misprices back
`inv_047_FablesLedgerV2`	FablesLedgerV2	High	behavioural	Check passed	13.7s	unstake is never pausable, so a holder can always stop routing fees to the distributor.	Holders stuck r distributor, or s the staked cut
`inv_048_FablesFeeEngine`	FablesFeeEngine	Low	behavioural	Check passed	8.3s	No hook uses return-delta flags, remove-liquidity or donate callbacks, or hookData.	Unexpected cal let the hook alt amounts.
`inv_049_FablesLedgerV2`	FablesLedgerV2	High	formula	Check passed	7.7s	`FablesLedgerV2._poolClaimFee[poolId]` (bps, history, allTimeMin): set by ADMIN via `FablesLedgerV2.setClaimFee` and by REVENUE_OPS via `FablesLedgerV2.setClaimFeeBps`, bounded by `LedgerLibV2.recordClaimFee` to `bps <= MAX_CLAIM_FEE_BPS` (2000), bps != 0 requires a non-zero recipient, plus `_checkKey`; each call pushes (fromTs, effective bps) to history and lowers allTimeMin if smaller. setClaimFeeBps reverts while claimFeeRecipient == 0. Read by	The rate decide of unstaked LP recipient at eac force a sync, sc whatever rate i synced.

ID	Contract	Crit.	Type	Verdict	Time	Invariant statement	Impact if viola
						`LedgerLibV2.creditFees` -> `LedgerLibV2.claimFeeFor` (rate = min over history entries since the range's `claimFeeSyncIndex`, starting from current bps, walking at most 32 entries, else min with allTimeMin; 0 if sinceIndex == 0 or recipient == 0) and `LedgerLibV2._split` (claim cut c = net_unstaked_fee * rate / 10000, rounded down, booked to claimFeeOwed); `FablesLedgerV2._effectiveClaimFee` compared to the user's maxFeeBps in `claimFees` / `withdrawAndClaim`.	
`inv_050_FablesLedgerV2`	FablesLedgerV2	High	behavioural	Check passed	8.8s	`FablesLedgerV2.claimFeeRecipient`: set by ADMIN via `FablesLedgerV2.setClaimFee`, bounded to: cannot return to address(0) once non-zero, `!= address(this)`, `!= poolManager`, `!= distributor`, `!= ffFeeSink`; otherwise any address. Before writing, only the currencies listed in `settleFirst` are swept to the old recipient. Read by `LedgerLibV2.sweepClaimFee` / `FablesNativeETHV2._sweepClaimFee` (destination of all booked `claimFeeOwed[currency]`, native paid as WETH; zero = no-op), `LedgerLibV2._split` and `LedgerLibV2.claimFeeFor` (zero recipient = rate 0), `FablesLedgerV2.setClaimFeeBps`, `FablesLedgerV2.setFFFeeSink` and `LedgerLibV2.validateDistributor` (exclusions).	Changing the r every currency that currency's revenue to the permissionless
`inv_051_FablesLedgerV2`	FablesLedgerV2	High	behavioural	Check passed	17.6s	`FablesLedgerV2.distributor` and its cached `FablesLedgerV2.distributorFF`: set by ADMIN via `FablesLedgerV2.setDistributor` after sweeping `settleFirst` currencies; bounded by `LedgerLibV2.validateDistributor` and `_checkSinkTarget` to zero (clears; distributorFF = 0) or a contract with code, `!= ffFeeSink`, `!= claimFeeRecipient`, `!= address(this)`, `!= poolManager`, whose `ff()` returns non-zero (cached as distributorFF). Read by `LedgerLibV2.sweepDistributor` (destination of `distributorOwed[currency]` unless currency == distributorFF, which goes to ffFeeSink; zero destination = accrual waits), `FablesNativeETHV2._sweepDistributor` (native pot wrapped to WETH and sent to distributor), `LedgerLibV2.recordClaimFee` and `FablesLedgerV2.setFFFeeSink` (exclusions).	The distributor staked-share re permissionless in currencies nc new distributor
`inv_052_FablesLedgerV2`	FablesLedgerV2	High	behavioural	Check passed	11.2s	`FablesLedgerV2.ffFeeSink`: set by ADMIN via `FablesLedgerV2.setFFFeeSink` after sweeping `settleFirst` currencies; bounded to zero, or a non-zero address `!= distributor`, `!= claimFeeRecipient`, `!= address(this)`, `!= poolManager`. Read by `LedgerLibV2.sweepDistributor` (destination of `distributorOwed[distributorFF]`, i.e. FABLES-denominated staked-share fees; zero = accrual waits), `LedgerLibV2.recordClaimFee` and `LedgerLibV2.validateDistributor` (exclusions). The native override `FablesNativeETHV2._sweepDistributor` does not read it.	The sink receiv denominated st next permissior
`inv_053_FablesFeeEngine`	FablesFeeEngine	Medium	behavioural	Check passed	13.9s	Shared state pool word `_word[id]` (FablesFeeEngine): written by `configurePool` (bases, floor, cap), `pokeFee` (pokes + expiry; also `_meta.lastRaiseAt` on a raise), `clearPoke` (clears pokes), `setDiscount` (ORs bloom bits), `setDiscountFilter` (replaces bloom and seed), `setCFActive` (cfOn bit) and `killCF` (cfOn false). CF word `_cf[id]` is written by `configureCF` (full rewrite incl. anchor at current tick; only while off), `setCFActive(true)` (`cfRearm` at current tick), `killCF` (k = 0) and by the `FablesFeeFull` beforeSwap fallback (tLast/anchor update when `dt >= period`, on any swap).	One writer clob another (e.g. bl a reconfigure), CF anchor.
`inv_054_FablesLedgerV2`	FablesLedgerV2	Medium	behavioural	Check passed	8.8s	Shared state pause timers: `pausedUntil` is set to `now + duration` or lifted to 0 by `setPaused(duration <= 7 days)`, and only extended by `pause(duration)` (reverts if `now + duration <= pausedUntil`); `pausedUntilOf[poolId]` likewise by `setPoolPaused` (set/lift) and `pausePool` (extend only). `pausedFor(poolId)` gates `deposit`, `withdrawAndClaim`, `claimFees`, `stake`, and (via range key) `transfer`/`transferFrom`; hook-wide `paused()` alone gates `sweepDistributor`/`sweepClaimFee`. `withdraw`, `unstake` and `syncRanges` read neither.	A pause blockir freeze being sh
`inv_055_FablesLedgerV2`	FablesLedgerV2	Medium	behavioural	Check passed	8.8s	Native-ETH deposit accounting: on `currency0 == address(0)` pools `deposit` requires `msg.value`	Refunding ETH (e.g. stranded

ID	Contract	Crit.	Type	Verdict	Time	Invariant statement	Impact if viola
						>= amount0Max`, stores `address(this).balance - msg.value` in the transient `_FOREIGN_NATIVE_SLOT`, the unlock settles `owed` currency0 via `poolManager.settle{value: owed}()` from the hook balance (bounded by `amount0Max` through `_checkBound`), and `_refundNative` sends `max(0, address(this).balance - pre)` to `msg.sender` via a call that reverts on failure. `receive()` accepts ETH only from `poolManager`; native revenue sweeps take ETH to the hook and wrap it to WETH in the same call.	under-refundin
`inv_056_FablesLedgerV2`	FablesLedgerV2	Medium	behavioural	Check passed	9.2s	`FablesLedgerV2.pausedUntil` (hook-wide): set by ADMIN via `FablesLedgerV2.setPaused` (duration <= MAX_PAUSE = 7 days; 0 sets it to 0; may shorten) and by PAUSER via `FablesLedgerV2.pause` (1 <= duration <= 7 days, new until = now + duration must exceed the current pausedUntil; no cap on how many times it can be extended). Read by `FablesLedgerV2.paused` / `FablesLedgerV2.pausedFor` -> `whenNotPaused` (`sweepDistributor`, `sweepClaimFee`), `whenPoolNotPaused` (`deposit`, `claimFees`, `withdrawAndClaim`, `stake`) and `whenRangeNotPaused` (`transfer`, `transferFrom`). Not read by `withdraw`, `unstake`, `syncRanges`, or the internal sweeps run by `setClaimFee`/`setDistributor`/`setFFFeeSink` via `_settleSinks`.	A running paus receipt transfer permissionless repeated PAUSI freeze running
`inv_057_FablesLedgerV2`	FablesLedgerV2	Medium	behavioural	Check passed	24.0s	`FablesLedgerV2.pausedUntilOf[poolId]` (per-pool): set by ADMIN via `FablesLedgerV2.setPoolPaused` (duration <= 7 days, 0 lifts, may shorten) and by PAUSER via `FablesLedgerV2.pausePool` (1..7 days, only extends past the current value), both requiring `_checkKey(key)`. Read by `FablesLedgerV2.pausedFor` -> `whenPoolNotPaused` (`deposit`, `claimFees`, `withdrawAndClaim`, `stake`) and `whenRangeNotPaused` (`transfer`, `transferFrom` via the range's stored key; an unknown range resolves to the zero key). Not read by sweeps, `withdraw`, `unstake`, `syncRanges`.	Freezes claims, deposits of one withdraw stay
`inv_058_FablesLedgerV2`	FablesLedgerV2	Medium	behavioural	Check passed	25.3s	`FablesLedgerV2.ballotGate`: set by ADMIN via `FablesLedgerV2.setBallotGate`, bounded to zero, or a contract with code that answers `ballotBacks`, `isGaugeEarning`, `gaugeWeightNow` without reverting and whose `ve()` escrow (non-zero, with code) returns `voter() == gate`. Read by `FablesLedgerV2.stake` (reverts GaugeNotEarning if gate == 0 or `isGaugeEarning(poolId)` is false; compares `gaugeWeightNow` to the caller's minGaugeWeight) and `FablesLedgerV2.syncRanges` (reverts BallotGateUnset if zero; requires `ballotBacks(msg.sender, tokenId, poolId)` per staked range).	The gate decid (redirecting fut which voters ca gate closes bot open.
`inv_059_FablesStableFee`	FablesStableFee	High	formula	Check passed	15.8s	`FablesStableFee._words[id].cfg` k / optimalFeeE6 / targetMultiplier / referenceSqrtPriceX96 and `FablesHookCore.maxFee[id]`: set by ADMIN via `FablesStableFee.configureStablePool` (once per pool: reverts if the reference is already non-zero; dynamic-fee key, hooks == this, native only on native builds; `k != 0` with no upper bound, `optimalFeeE6 <= 1e4`, `1 <= targetMultiplier <= 100`, reference within StablePair's bounded v4 range; initial `sqrtPriceX96` passed straight to `poolManager.initialize` with no link to the reference; `maxFee = STABLE_MAX_FEE`) and via `FablesStableFee.updateFeeConfig` (k, optimalFeeE6, targetMultiplier with the same bounds; reference kept; fee state reset). Read by `FablesStableFee._stableFee` -> `_getFee` / `_calculateDecayingFee` / `_targetFee` from the `FablesStableHookLite` fallback and `FablesStableHookLiteTyped.beforeSwap`, and by `getFee`, `currentFee`, `feeFor`, `feeConfig`.	k sets how fast block; optimalF set the in-band target; togethe swap fee (capp
`inv_060_FablesStableHookLite`	FablesStableHookLite	High	formula	Check passed	11.0s	Stable reference price (`referenceSqrtPriceX96` in `FablesStableFee._words[id].cfg`) and `FablesStableHookLite.referenceUpdatedAt[id]`: moved by STABLE_KEEPER via `FablesStableHookLite.updateReferencePrice`, bounded to pool initialized, `block.timestamp >= referenceUpdatedAt + REFERENCE_UPDATE_INTERVAL` (60 s), new	The reference charged and hc minute compou per hour, and e decaying fee st

ID	Contract	Crit.	Type	Verdict	Time	Invariant statement	Impact if viola
						reference within the validated range, and `calculatePriceRatioX96(new, previous) >= MIN_REFERENCE_RATIO_X96` (0.999 in Q96); k/optimalFeeE6/targetMultiplier and the whitelist bits kept; fee state reset (decaying fee UNDEFINED, cached price 0). No cumulative bound across calls. Read by `FablesStableFee._getFee` (priceRatio = ratio of AMM sqrt price to reference; picks inside-band fee vs close/far boundary fees and which side pays 0), `_decayStartFee`, `_targetFee` (floor beyond ratio 0.9), and views `getFee` / `currentFee` / `feeFor` / `feeConfig`.	
`inv_061_FablesStableFee`	FablesStableFee	High	formula	Check passed	8.2s	`FablesStableFee._entry[id][account]` and the 32-bit Bloom/seed in `FablesStableFee._words[id].cfg` bits [216,256): set by DISCOUNT_ADMIN via `FablesStableFee.setDiscount` (account != 0, bps <= 5000; for bps != 0: dirs in {1,2,3}, expiry in (now, now + 90 days]; pool initialized; minFee unbounded; bps == 0 deletes) and `FablesStableFee.setDiscountFilter` (any uint8 seed, Bloom rebuilt from listed accounts holding an entry); deleted by PAUSER via `FablesStableFee.revokeDiscount` (no pool check). Read by `FablesStableFee._stableFee` -> `FablesStableFee._discounted` when bloom != 0 and fee != 0: fee = min(fee, max(ceil(fee*(1 - min(bps,50%))), minimumFee, entry minFee)), and by `FablesStableFee.feeFor`.	Listed senders stable pools, flo minimum fee a
`inv_062_FablesLedgerV2`	FablesLedgerV2	Medium	behavioural	Check passed	25.2s	Fee crediting in a share change occurs at post-mutation totals, which the quote says is safe only because each caller's earlier _syncRange drained the fees.	Any non-zero fe modifyLiquidity between the tw via a hook callb split at the post the new deposi
`inv_063_FablesStableFee`	FablesStableFee	Medium	behavioural	Check passed	19.1s	Shared state stable fee words `_words[poolId]`: `state` (decaying fee [0,40), cached sqrtPrice [40,200), clock block [200,240), minimum fee [240,256)) is written by the swap path `_stableFee` (only on the first swap of a clock block or after a reset; preserves min-fee bits), by `_resetFeeState` (from `configureStablePool`, `updateFeeConfig`, `updateReferencePrice`, and `setMinimumFee` when lowering) and by `setMinimumFee` (min-fee bits). `cfg` (k, optimalFeeE6, targetMultiplier, reference [0,216); bloom32/seed [216,256)) is written by `_writeFeeConfig` (keeps whitelist bits) and by the discount setters (whitelist bits).	A writer clobbe whitelist bits, o for remaining s
`inv_064_FablesHookCore`	FablesHookCore	High	behavioural	Check passed	7.7s	`AccessManaged.authority` of every hook build (set in the `FablesHookCore` constructor): replaceable only by the current authority via `AccessManaged.setAuthority` (AccessManager ADMIN through `updateAuthority`), with no bound on the new address; role-to-selector bindings, role grants and execution/admin delays live in the AccessManager and are changed by ADMIN. Read by the `restricted` modifier on every setter of `FablesFeeEngine`, `FablesLedgerV2`, `FablesStableFee` and `FablesStableHookLite`. The hooks have no proxy or upgrade path; `poolManager`, `weth` and `USE_ARB_SYS` are immutables.	Whoever contro every restricted revenue sinks a
`inv_065_FablesFeeEngine`	FablesFeeEngine	High	bound:le	Check passed	16.4s	ADMIN is documented as unable to move LP principal, set a fee above 10%, or un-configure a pool.	A path that retu pips (e.g. a stal category chang or minimumFee overcharge swa documented ce
`inv_066_FablesLedgerV2`	FablesLedgerV2	High	bound:le	Check passed	9.5s	The claim-fee rate charged on unstaked LP fees is at most 20% (`MAX_CLAIM_FEE_BPS` = 2000), enforced in `LedgerLibV2.recordClaimFee` for both `FablesLedgerV2.setClaimFee` (ADMIN) and `FablesLedgerV2.setClaimFeeBps` (REVENUE_OPS); the rate applied in `LedgerLibV2._split` is `claimFeeFor` over recorded history entries.	A skim above 2 exceed the doc limit.
`inv_067_FablesLedgerV2`	FablesLedgerV2	High	behavioural	Check passed	24.6s	`FablesLedgerV2.withdraw` and `FablesLedgerV2.unstake` carry no pause modifier and no role gate (withdraw auto-unstakes the shortfall), whereas `withdrawAndClaim`, `claimFees`, `deposit`, `stake` and receipt transfers are gated by `whenPoolNotPaused` / `whenRangeNotPaused`. The ledger NatSpec documents this as a design property.	Any path by wh setting (ballot g an external sink withdraw/unsta principal agains property.
`inv_068_FablesHookCore`	FablesHookCore	Medium	behavioural	Check passed	11.6s	`FablesHookCore.maxFee[id]` is written only by `FablesFeeEngine.configurePool` (= cap, cap >= 1)	If maxFee retur would revert in

ID	Contract	Crit.	Type	Verdict	Time	Invariant statement	Impact if viola
						and `FablesStableFee.configureStablePool` (= STABLE_MAX_FEE), never to 0; `FablesLedgerV2._checkKey` and `FablesFeeEngine.initializePool` treat a non-zero value as "configured". The core NatSpec states the marker is never reset to 0.	trapping that p fees.
`inv_069_FablesLedgerV2`	FablesLedgerV2	Medium	bound:le	Check passed	20.6s	Each pause call extends a freeze by at most `MAX_PAUSE` (7 days) from the call's timestamp: `FablesLedgerV2.pause` / `pausePool` (PAUSER) require 1 <= duration <= 7 days and until > current value; `setPaused` / `setPoolPaused` (ADMIN) require duration <= 7 days. The cap is per call; the code sets no limit on the number of successive extensions.	Freezes of clain longer than 7 d PAUSER calls; a pausedUntil fur one call would per-call limit.
`inv_070_FablesFeeEngine`	FablesFeeEngine	High	bound:le	Check passed	536.7s	A FEE_POKER poke is valid for at most `MAX_POKE_TTL` (72 h) and a raising poke may move the resolved keeper fee up by at most the category `raiseStep`, no sooner than `RAISE_INTERVAL` (60 s) after the pool's last raise (`FablesFeeEngine.pokeFee`, `FablesFeeEngine._raises`, `FablesFeeEngine._meta.lastRaiseAt`).	A raise larger th raises within 60 clearPoke/confi non-raising pok would exceed t bound.
`inv_072_LedgerLibV2`	LedgerLibV2	Medium	behavioural	Check passed	8.4s	Owed accrual is saturating, with tokens above 2^127 units unsupported, because a revert would block the holder's withdraw.	Accrual overflo or saturation si amounts, break holder fairness.
`inv_073_LedgerHandler`	LedgerHandler	Low	monotonicity:ge	Check passed	8.7s	`LedgerHandler.callCount` is a cumulative counter that only increases, by exactly 1 per handler call, and is never decreased or reset by any handler function.	Test-harness ca unreliable; low
`inv_074_ConservationHandler`	ConservationHandler	Low	monotonicity:ge	Check passed	7.5s	`ConservationHandler.callCount` and `ConservationHandler.slackGrowths` only increase, by at most 1 per handler call.	Test bookkeepi
`inv_075_IsolationHandler`	IsolationHandler	Low	monotonicity:ge	Check passed	31.1s	`pausesLanded` is a cumulative counter that only increases (by one per successful `pausePool` or `liftPool`).	Test ghost cour the actual paus
`inv_076_EngineInvariantHandler`	EngineInvariantHandler	Low	monotonicity:ge	Check passed	17.3s	`fieldViolations` is a cumulative counter that only increases (via ++) and stays 0 if the hook preserves non-poke fields of the pool word.	A nonzero valu changed pool-w have changed.
`inv_077_AccWrapHarness`	AccWrapHarness	High	conservation:eq	Check passed	26.0s	`_ranges[ID].totalShares` equals the sum of `balanceOf` over all accounts; `join` and `leave` change both by the same amount.	Share accounti are misallocate
`inv_078_AccWrapHarness`	AccWrapHarness	High	conservation:le	Check passed	8.3s	`_ranges[ID].totalStaked` equals the sum of `_pos[who].staked` over all accounts, and `_ranges[ID].totalStaked` is at most `_ranges[ID].totalShares`. Each account's staked amount is at most its `balanceOf`. `unstake` does not check the amount against the staked balance, so it relies on checked arithmetic.	Staked totals e the shares, whi distribution.
`inv_005_FablesFeeEngine`	FablesFeeEngine	High	bound:le	Violated	22.9s	The LP fee a swap pays (the fee returned with OVERRIDE_FEE_FLAG by every build's beforeSwap entry) must be <= the pool cap and <= 100,000 pips (10%). Fee-engine builds: `FeeRef.keeperFee` clamps to `w.cap()` and `FeeRef.resolve` clamps again after the CF surcharge; `configurePool` requires `cap <= capMax` and `setCategory` requires `capMax <= ABSOLUTE_MAX_FEE` (100_000). Stable builds: `_toPips` caps at `STABLE_MAX_FEE` (100_000) and `configureStablePool` writes `maxFee[id] = STABLE_MAX_FEE`; the minimum fee (uint16, <= 65,535 pips) and discount must not push the charged fee above the cap.	Swappers overc documented 10 the pool cap; b set a fee above
`inv_008_FablesFeeEngine`	FablesFeeEngine	High	bound:le	Violated	20.9s	FEE_POKER can turn the continuation fee on or re-anchor it, only at a tick inside the range the call gives and by its deadline, or off.	A semi-trusted undercharges s documented lir
`inv_033_FablesStableFee`	FablesStableFee	Medium	formula	Inconclusive	78.9s	Stable-build decay target below the 0.9 window: the formula reads and stores the real price, as StablePair; once the ratio is below 0.9 the target is max(far - close*targetMultiplier/100, edgeFar - edgeClose*targetMultiplier/100) with edge values computed at MIN_FEE_PRICE_RATIO_X96 (0.9 in Q96). The floor is skipped (never reverts) when edgeClose <= 0 or edgeFar <= edgeCloseShare. Inputs: optimalFeeE6, targetMultiplier set by ADMIN; price by any swapper; reference by ADMIN/STABLE_KEEPER.	Decay target fa capped target, swaps.
`inv_040_FablesFeeEngine`	FablesFeeEngine	Medium	authorization	Check passed	24.7s	SEEDER may only initialize a pool the admin has already configured (FablesHook builds), at the price it passes: `FablesFeeEngine.initializePool(key, sqrtPriceX96)` requires `maxFee[id] != 0` (set only	Unconfigured p seeder changes

ID	Contract	Crit.	Type	Verdict	Time	Invariant statement	Impact if viola
						by `configurePool`) and native currency0 only on native builds; the hook places no bound on `sqrtPriceX96`. SEEDER must not be able to configure pools or change existing ones (v4 reverts re-initialization of an initialized pool).	
`inv_071_FablesStableHookLite`	FablesStableHookLite	**High**	bound:le	**Check passed**	16.8s	Each updateReferencePrice requires ratio >= 0.999 and setMinimumFee takes a uint16 (at most 65,535 pips).	A reference mo minute (e.g. via ordering of argu `configureStabl writing the refe documented ke
`inv_010_FablesFeeEngine`	FablesFeeEngine	**Medium**	behavioural	**Untranslatable**	—	Switching the CF on via `FablesFeeEngine.setCFActive(poolId, true, minTick, maxTick, deadline)` must require the current slot0 tick inside [minTick, maxTick] and block.timestamp <= deadline, then re-anchor at that tick (`FeeRef.cfRearm`); `configureCF` must revert `CFActive` while the CF is on, so its anchor (set at the current tick) is replaced on switch-on. (Fix for pre-audit finding 5.1.2 'manipulable spot anchor'.) Observation: switching off has no tick/deadline precondition and `setCFActive(true)` also works while the CF is already on (re-anchoring).	Whoever lands switch-on can s anchor, chargin to cfCap extra f

OLYMPIX
Proactive Web3 smart contract security

© 2026 Olympix Inc. All rights reserved. · Prepared for Fables · Session `3ad548de-4adf-4c21-bfd4-80141c63f9f3` · November 11, 2026 UTC · Every verdict in this report is reproducible from the Foundry artifacts shipped alongside it, within the scope each check declares.